

# Lab3 MaskGIT for Image Inpainting Report

Deep Learning, NYCU

朱驛庭, 111550093

## 1. Introduction

In this lab, we implement **MaskGIT** [1] for the image inpainting task on  $64 \times 64$  cat face images. The pipeline follows a two-stage approach:

1. **Stage 1 (VQGAN)** [2]: A pretrained VQGAN encoder compresses  $64 \times 64 \times 3$  images into a discrete latent representation of 256 tokens ( $16 \times 16$  spatial, flattened), each mapped to one of 1024 codebook entries via vector quantization [4]. This stage is frozen and not modified.
2. **Stage 2 (Bidirectional Transformer)**: A transformer trained from scratch to predict masked tokens given unmasked context, using the Masked Visual Token Modeling (MVTM) mechanism. At inference, iterative decoding with mask scheduling progressively fills in masked regions.

The report is structured as follows: Section 2 details the implementation of multi-head self-attention, MVTM training, and iterative decoding for inpainting. Section 3 presents the experimental results including iterative decoding visualizations for all three mask scheduling functions and the best FID [3] score. Section 4 discusses training strategies, hyperparameter choices, and observations.

## 2. Implementation Details

### 2.1. Multi-Head Self-Attention (TODO1)

The core attention mechanism is implemented from scratch without using any predefined function like `torch.nn.MultiheadAttention`, following the scaled dot-product attention formulation from [5]:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V$$

where  $d_k = d_v = 768/16 = 48$  is the per-head dimension. The implementation consists of:

- Four linear projections:  $W_Q, W_K, W_V \in \mathbb{R}^{768 \times 768}$  for query, key, value, and  $W_O \in \mathbb{R}^{768 \times 768}$  for the output projection.
- The input  $(B, 256, 768)$  is projected to  $Q, K, V$ , reshaped to  $(B, 16, 256, 48)$  for multi-head processing.
- Attention weights are computed, followed by dropout ( $p = 0.1$ ), then multiplied by  $V$ .
- Heads are concatenated and projected through  $W_O$  back to dimension 768.

```
1 class MultiHeadAttention(nn.Module):
2     def __init__(self, dim=768, num_heads=16,
3                   attn_drop=0.1):
4         super().__init__()
5         self.N_heads = num_heads
6         self.d_k = dim // num_heads # 48
7         self.W_q = nn.Linear(dim, dim)
8         self.W_k = nn.Linear(dim, dim)
9         self.W_v = nn.Linear(dim, dim)
10        self.W_out = nn.Linear(dim, dim)
11        self.attn_drop = nn.Dropout(attn_drop)
12
13    def forward(self, x): # (B, 256, 768)
14        B, N, D = x.shape
15        Q = self.W_q(x).view(B, N, self.N_heads,
16                               self.d_k).transpose(1, 2)
17        K = self.W_k(x).view(B, N, self.N_heads,
18                               self.d_k).transpose(1, 2)
19        V = self.W_v(x).view(B, N, self.N_heads,
20                               self.d_k).transpose(1, 2)
21        # (B, 16, 256, 48)
22        attn = torch.softmax(
23            Q @ K.transpose(-2, -1)
24            / math.sqrt(self.d_k), dim=-1)
25        attn = self.attn_drop(attn)
26        out = (attn @ V).transpose(1, 2)
27        out = out.contiguous().view(B, N, -1)
28        return self.W_out(out)
```

Within the **Encoder** block, residual connections and layer normalization are applied externally: `x = LayerNorm(x + Dropout(Attention(x)))`.

## 2.2. Stage 2 Training: MVTM (TODO2)

### 2.2.1 Encoding to Discrete Tokens

Given an input image  $x$ , we pass it through the frozen VQGAN encoder to obtain the quantized latent and the corresponding codebook indices:

```
1 @torch.no_grad()
2 def encode_to_z(self, x):
3     zq, z_indices, _ = self.vqgan.encode(x)
4     B = zq.size(0)
5     z_indices = z_indices.view(B, -1) # (B,
6     ↪ 256)
7     return zq, z_indices
```

### 2.2.2 Mask Scheduling Functions

Three scheduling functions  $\gamma(r)$  map a ratio  $r \in [0, 1]$  to a mask rate  $\in (0, 1]$ :

$$\begin{aligned}\gamma_{\text{linear}}(r) &= 1 - r \\ \gamma_{\text{cosine}}(r) &= \cos\left(\frac{\pi r}{2}\right) \\ \gamma_{\text{square}}(r) &= 1 - r^2\end{aligned}$$

During training,  $r \sim \text{Uniform}(0, 1)$  is sampled independently per sample; during inference,  $r = t/T$  follows the iteration schedule.

### 2.2.3 Training Forward Pass

The forward pass implements MVTM: for each sample in the batch, a random ratio  $r$  is drawn, producing a per-sample mask rate  $\gamma(r)$ . The corresponding number of tokens are randomly masked with `mask_token_id = 1024`:

```
1 def forward(self, x):
2     _, z_indices = self.encode_to_z(x)
3     masked_indices = z_indices.clone()
4     B = z_indices.size(0)
5     for i in range(B):
6         r = torch.rand(1).item()
7         mask_rate = self.gamma(r)
8         num_mask = max(1,
9             ↪ math.ceil(mask_rate * 256))
10        mask_pos =
11            ↪ torch.randperm(256)[:num_mask]
12        masked_indices[i, mask_pos] = 1024
13    logits = self.transformer(masked_indices)
14    return logits, z_indices # (B, 256, 1025),
15    ↪ (B, 256)
```

The loss is the standard cross-entropy between the transformer’s logits and the ground-truth codebook in-

dices over all 256 token positions:

$$\mathcal{L} = \text{CrossEntropy}(\hat{y}, y) = -\frac{1}{N} \sum_{i=1}^N \log \hat{y}_{i, y_i}$$

### 2.2.4 Training Strategy

We use **AdamW** optimizer with learning rate  $10^{-4}$ , batch size 64, and gradient accumulation of 2 steps (effective batch size 128). Three learning rate schedulers were compared:

- **CosineAnnealingLR**: Decays LR following a cosine curve over epochs.
- **ReduceLROnPlateau**: Halves LR when validation loss plateaus (patience=10).
- **OneCycleLR**: Warms up to max LR (10% of training) then cosine anneals per batch.

All models were trained for 150 epochs. The best checkpoint is saved based on the lowest validation loss.

## 2.3. Inference: Iterative Decoding for Inpainting (TODO3)

At inference, given a masked image and its binary mask, the pipeline:

1. Encodes the image to 256 codebook indices via VQGAN.
2. Sets masked positions (where `mask = 1`) to `mask_token_id = 1024`.
3. Iteratively decodes for  $T$  steps. At each step  $t$ :
  - The transformer predicts logits over all positions.
  - Softmax probabilities determine the most likely codebook entry per token.
  - Gumbel noise with temperature  $\tau = 4.5 \times (1 - r)$  is added to confidence scores for diversity.
  - Non-masked (original) positions are assigned confidence  $= +\infty$ .
  - The  $n = \lceil \gamma(r) \times M \rceil$  lowest-confidence positions are re-masked, where  $M$  is the initial mask count.
  - Original (unmasked) token values are re-stored.
4. At the sweet spot step  $t^*$ , the final predicted indices are decoded back to a  $64 \times 64$  image via VQGAN.

```

1 @torch.no_grad()
2 def inpainting(self, z_indices, mask_bc,
3               mask_b, ratio, mask_num):
4     logits = self.transformer(z_indices)
5     logits = torch.softmax(logits, dim=-1)
6     prob, pred = logits.max(dim=-1)
7     # Gumbel noise for stochastic confidence
8     g = -torch.log(-torch.log(
9         torch.rand_like(prob)))
10    temperature = 4.5 * (1 - ratio)
11    confidence = prob + temperature * g
12    confidence[~mask_b] = float("inf")
13    n = math.ceil(self.gamma(ratio) * mask_num)
14    _, idx = confidence.sort(dim=-1)
15    mask_bc = torch.zeros_like(mask_b)
16    mask_bc.scatter_(1, idx[:n], True)
17    pred[mask_bc] = self.mask_token_id
18    pred[~mask_b] = z_indices[~mask_b]
19    return pred, mask_bc

```

## 3. Experimental Results

### 3.1. Part 1: Iterative Decoding Visualization

We visualize the iterative decoding process for all three mask scheduling functions (cosine, linear, square) with  $T = 15$  and sweet spot  $t^* = 14$ , using the best checkpoint (OneCycleLR, 150 epochs).

#### 3.1.1 Mask Scheduling Functions

Figure 1 shows the three scheduling functions  $\gamma(r)$ . The cosine schedule aggressively unmask early tokens while preserving more masks in later iterations, providing the model with richer context during the critical final steps. The linear schedule unmask at a constant rate. The square schedule retains a high mask rate initially and accelerates unmasking toward the end.

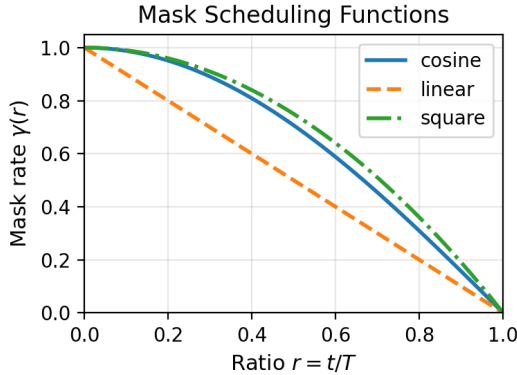
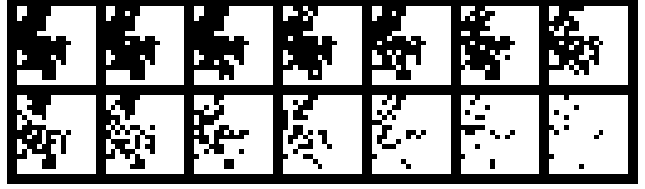


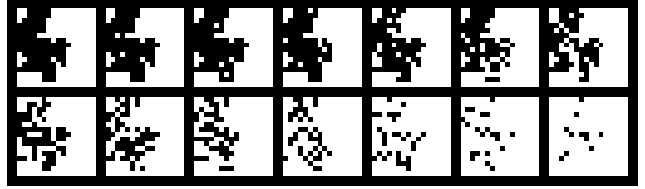
Figure 1. Mask scheduling functions  $\gamma(r)$ : fraction of tokens remaining masked as a function of the normalized iteration ratio  $r = t/T$ .

#### 3.1.2 Mask Evolution in Latent Domain

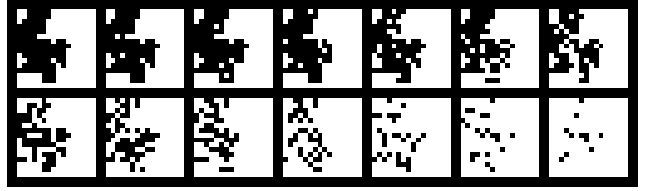
Figure 2 compares the mask evolution in the  $16 \times 16$  latent space across the three scheduling functions. White pixels indicate masked (unknown) tokens; black pixels indicate predicted (unmasked) tokens.



(a) Cosine schedule



(b) Linear schedule



(c) Square schedule

Figure 2. Mask evolution in the  $16 \times 16$  latent domain over 14 iterations ( $t = 1$  to  $t = 14$ , left to right). The cosine and square schedules unmask more aggressively in early iterations, while linear progresses at a constant rate.

#### 3.1.3 Predicted Image Evolution

Figure 3 shows the corresponding decoded images at each iteration. The first frame is the masked input; subsequent frames show progressive refinement as more tokens are predicted and decoded through the VQGAN decoder.

### 3.2. Part 2: Best FID Score

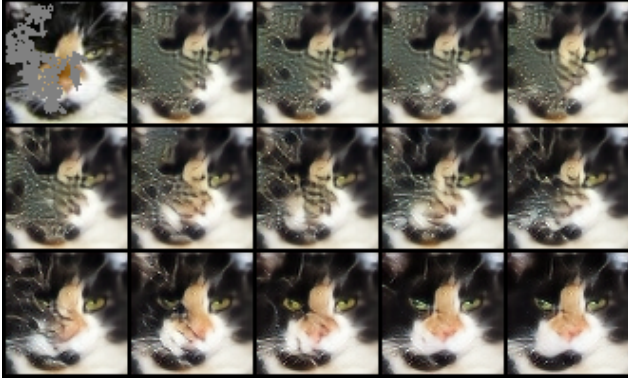
#### 3.2.1 FID Score Screenshot

Our best FID score of **29.11** was achieved with the following settings:

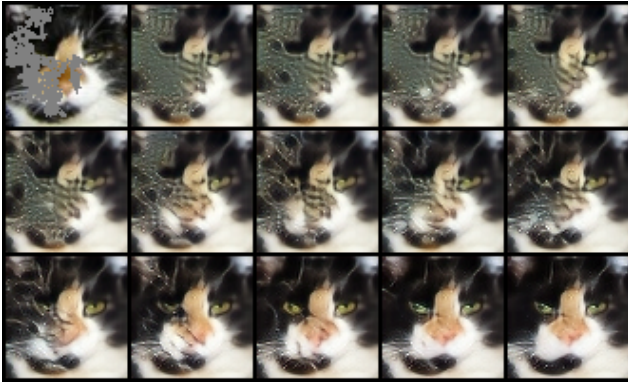
- **Checkpoint:** OneCycleLR best checkpoint (150 epochs, batch size 64)
- **Mask function:** cosine
- **Total iterations:**  $T = 10$



(a) Cosine schedule



(b) Linear schedule



(c) Square schedule

Figure 3. Predicted images over 15 frames (masked input  $\rightarrow$  iterations 1–14, left to right, top to bottom). All three schedules converge to similar final quality, but the cosine schedule produces recognizable structure earlier.

- **Sweet spot:**  $t^* = 9$

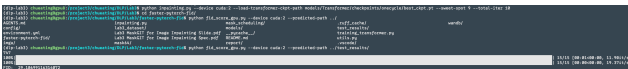


Figure 4. FID score screenshot: **29.11** on 747 test images.

### 3.2.2 FID Comparison Across Scheduling Functions

| Mask Function | $T$ | $t^*$ | FID $\downarrow$ |
|---------------|-----|-------|------------------|
| cosine        | 10  | 9     | <b>29.11</b>     |
| linear        | 10  | 9     | 30.25            |
| square        | 10  | 9     | 29.73            |

Table 1. FID scores for different mask scheduling functions. All use OneCycleLR checkpoint with  $T = 10$ ,  $t^* = 9$ .

### 3.2.3 Masked Images vs. Inpainting Results



Figure 5. Masked input images (left of each pair) and MaskGIT inpainting results (right). The model successfully fills in the gray masked regions with plausible cat face textures.

## 4. Discussion

### 4.1. Training Strategy Comparison

We compared three learning rate schedulers under identical conditions (AdamW,  $\text{lr} = 10^{-4}$ , 150 epochs, batch size 64, gradient accumulation 2).

- **OneCycleLR** achieved the best validation loss and FID score. The warm-up phase helps the model explore the loss landscape before settling, and the per-batch LR updates provide smoother optimization than per-epoch schedulers.
- **CosineAnnealingLR** performed reasonably but decayed LR to zero by the end, potentially cutting off useful learning in later epochs.
- **ReduceLROnPlateau** reduced LR too aggressively (factor=0.5, patience=10), causing the effective learning rate to drop to near-zero prematurely, resulting in stalled training.

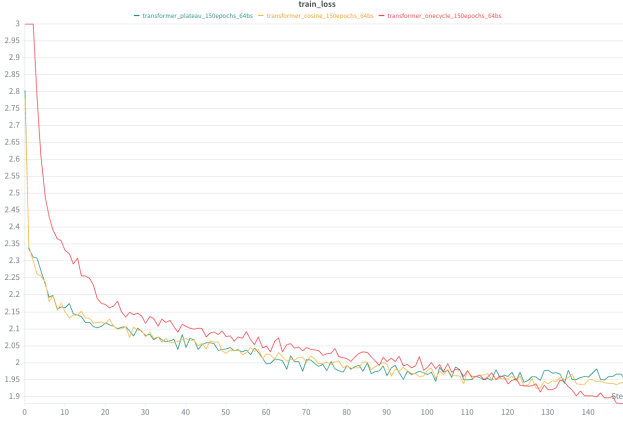


Figure 6. Training loss curves for three LR schedulers.



Figure 7. Validation loss curves for three LR schedulers.

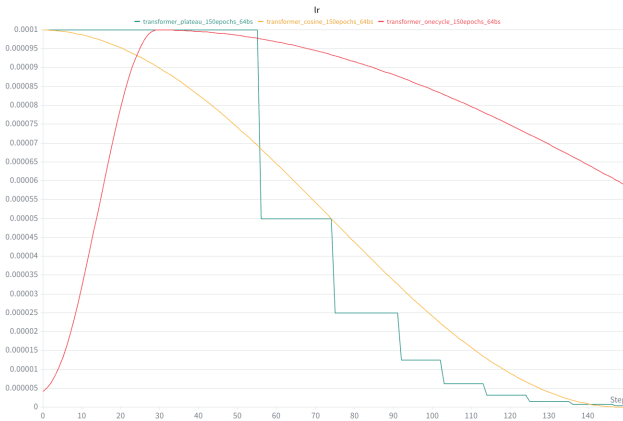


Figure 8. Learning rate schedules over training.

## 4.2. Sweet Spot and Total Iterations

We swept  $t^*$  (sweet spot) and  $T$  (total iterations) to find the optimal decoding configuration. Table 2

summarizes the results.

| $T$ | $t^*$ | FID ↓        |
|-----|-------|--------------|
| 10  | 5     | 76.98        |
| 10  | 6     | 55.60        |
| 10  | 7     | 40.86        |
| 10  | 8     | 33.03        |
| 10  | 9     | <b>28.94</b> |
| 15  | 10    | 44.99        |
| 15  | 11    | 38.30        |
| 15  | 12    | 32.83        |
| 15  | 13    | 31.58        |
| 15  | 14    | 29.02        |
| 20  | 14    | 40.70        |
| 20  | 16    | 33.28        |
| 20  | 18    | 31.18        |
| 20  | 19    | 30.60        |

Table 2. Sweep of total iterations  $T$  and sweet spot  $t^*$ . Using nearly all iterations ( $t^* = T - 1$ ) yields the best FID.

The clear trend is that using nearly all iterations ( $t^* = T - 1$ ) gives the best FID. Across all three settings,  $T = 10$  with  $t^* = 9$  achieves the lowest FID of 28.94. With larger  $T$ , the cosine schedule spreads unmasking more thinly across more iterations—each step unmasks fewer tokens, giving the model less context per prediction. This increases re-masking noise and error accumulation, resulting in higher FID despite more refinement steps. The  $T = 15$  results ( $t^* = 14 \rightarrow$  FID 29.02) are competitive but do not surpass  $T = 10$ .

## 4.3. Per-Sample Masking Diversity

An important implementation detail is sampling the mask ratio  $r$  *independently* for each sample in the batch, rather than sharing a single  $r$  across the entire batch. This provides the transformer with a more diverse distribution of masking conditions during training, which improves generalization and led to lower validation loss in our experiments.

As shown in Fig. 9, the run without per-sample  $r$  (green, shared  $r$  across the batch) exhibits highly volatile validation loss with large spikes throughout training, whereas per-sample  $r$  (orange) converges smoothly and achieves a consistently lower validation loss. The shared- $r$  approach causes entire batches to have the same masking difficulty, leading to high-variance gradients and unstable optimization.

## References

- [1] Huiwen Chang, Han Zhang, Lu Jiang, Ce Liu, and William T. Freeman. Maskgit: Masked generative image transformer. In *CVPR*, 2022. 1

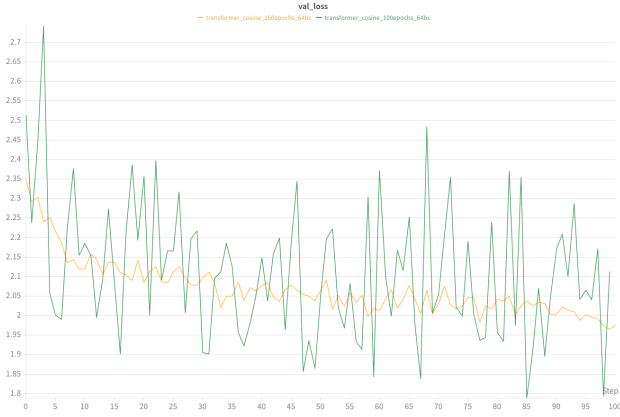


Figure 9. Validation loss with per-sample  $r$  (orange, smooth) vs. shared batch-level  $r$  (green, noisy). Per-sample masking diversity stabilizes training and yields lower final loss.

- [2] Patrick Esser, Robin Rombach, and Björn Ommer. Taming transformers for high-resolution image synthesis. In *CVPR*, pages 12873–12883, 2021. [1](#)
- [3] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. Gans trained by a two time-scale update rule converge to a local nash equilibrium. In *NeurIPS*, 2017. [1](#)
- [4] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *NeurIPS*, pages 6306–6315, 2017. [1](#)
- [5] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *NeurIPS*, 2017. [1](#)